Comparative Performance Analysis of Serial, OpenMP and CUDA Implementations of a Multi-Scale Frangi Pipeline for Retinal Vessel Segmentation in Diabetic Retinopathy Screening

Devik Nangia Roll No. 25 Reg. No. 240962372
Snehal Tyagi Roll No. 24 Reg. No. 240962368
Srijan Bera Roll No. 17 Reg. No. 240962300
Prathamesh Patil Roll No. 9 Reg. No. 240962052

Manipal Institute of Technology, Manipal
Parallel Computing Laboratory: Mini Project
(Submission 1: Proposal, Literature Survey, Objectives and Methodology)

***Abstract*—Diabetic retinopathy screening depends on fast and reliable segmentation of retinal vessels, and the multi-scale Frangi vesselness filter is a training-free classical method whose cost grows with image resolution and with the number of scales. This paper proposes a comparative performance analysis of serial C, OpenMP and CUDA implementations of a complete pipeline (green-channel extraction, CLAHE, multi-scale Frangi filtering, thresholding and morphological clean-up) evaluated on the DRIVE benchmark. A survey of twelve related works indicates that no earlier study profiles this full pipeline stage by stage across the three platforms. We describe the methodology, the per-stage parallel design (including naive and shared-memory tiled CUDA kernels), the measurement protocol and the metrics: Dice coefficient, speedup, efficiency, Karp–Flatt serial fraction, throughput and CPU–GPU crossover resolution. The work will test four hypotheses on OpenMP scaling, CUDA scaling with image size, the benefit of shared-memory tiling, and accuracy relative to lightweight deep models.**

***Index Terms*—parallel computing, OpenMP, CUDA, shared-memory tiling, Frangi filter, retinal vessel segmentation, diabetic retinopathy, performance analysis.**

I. Introduction

A. Problem Statement

Diabetic retinopathy (DR) damages the small blood vessels of the retina and is a leading preventable cause of vision loss. It is diagnosed from colour fundus photographs, and the first computational step in almost every automated screening system is segmenting the retinal vessels from the background. India had an estimated 89.8 million adults living with diabetes in 2024 [1], and screening camps capture fundus images in bulk, often on modest hardware and with a limited number of specialists available to read them.

The multi-scale Frangi vesselness filter [2] is a widely used classical method for this step. It needs no training data, behaves deterministically and can be written in plain C. Its cost, however, grows with image resolution and with the number of scales, because every pixel needs several Gaussian-derivative convolutions followed by an eigenvalue computation. Published parallel work on retinal vessels has either used different algorithms or accelerated isolated stages (Section II-E), and recent lightweight deep-learning work reports parameter counts and FLOPs rather than measured execution time on screening-class hardware.

Problem addressed. The aim is to implement the complete pipeline (green channel, CLAHE, multi-scale Frangi filter, morphological clean-up, thresholding) in serial C, OpenMP and CUDA, to verify that all three give equivalent segmentations on the DRIVE benchmark, and to explain, stage by stage and as image size grows, where each platform gains speed and where it is limited.

B. Motivation

- **Public-health scale.** Large diabetic populations and bulk image capture make throughput (images per second) a practical requirement, not only an academic metric.
- **Hardware reality.** Screening setups range from multicore laptops to low-cost NVIDIA GPUs. A study that compares OpenMP and CUDA on one pipeline tells a developer which hardware investment is worthwhile at which image size.
- **Gap in the literature.** The closest GPU works cover a contour-tracing method [3] and a standalone eigenvalue kernel [4]. We found no study of the full CLAHE, Frangi and morphology pipeline compared across serial, OpenMP and CUDA with per-stage profiling (Section II-E).
- **Learning value.** The pipeline contains compute-bound stages (eigenvalues), memory-bound stages (convolution, morphology) and an atomic-contention stage (histograms), so it exercises most of the ideas taught in the course.
- **Verifiable results.** DRIVE [5] is a standard benchmark with expert ground truth, so correctness can be checked and quality can be compared with published figures.

II. Literature Survey

A. Scope and Search Strategy

The survey covers twelve works. Ten were published between 2021 and 2026, and two earlier GPU papers [3], [4] are

included because they are the closest prior parallel implementations for retinal vessels. Sources were IEEE Xplore (IEEE Access), SpringerNature (Scientific Reports, BMC Bioinformatics, Journal of Real-Time Image Processing), Elsevier ScienceDirect (Computers in Biology and Medicine), SPIE (Journal of Electronic Imaging) and arXiv. Search terms combined retinal vessel segmentation, Frangi, vesselness, GPU, CUDA, OpenMP, lightweight network and real-time. The arXiv items [6]–[9] are preprints and are used only to show the most recent lightweight-network trend; they have not necessarily been peer reviewed. Foundational references for the Frangi filter, the DRIVE and STARE datasets, CLAHE, OpenMP, CUDA, the Karp–Flatt metric and the IDF statistics are cited where they are used.

B. Existing Approaches

1) Surveys and trends: Chen et al. [10] review deep-learning vessel segmentation, concentrating on network architectures, obstacles and future directions. Sule [11] reviews 110 papers from 2016 to 2021 and organises them by enhancement technique: optimisation algorithms, regularisation, pooling, activation functions, transfer learning and ensembles. Gojić et al. [6] give a shorter overview of recent deep methods with their reported metrics. All three are accuracy-centred: how fast a method runs on commodity hardware is not an organising theme.

2) Classical, vesselness-based methods: The Frangi filter [2] scores each pixel by how tube-like its Hessian eigenvalue structure is, at several Gaussian scales. Mahapatra et al. [12] propose an optimal improved Frangi-based multi-scale filter whose parameters are tuned by a modified enhanced-leader particle swarm optimisation (MELPSO), followed by adaptive weighted spatial fuzzy c-means clustering. This keeps the interpretable Frangi core and needs no labelled training data, but the parameter search and the clustering add computation, and parallel execution is not addressed.

3) Lightweight and efficient deep models: Galdran et al. [13] show that small, minimalistic networks can reach state-of-the-art vessel segmentation and run a cross-dataset study. Newer work targets deployment explicitly: LVS-Net [7] (0.71 M parameters, 29.60 GFLOPs), LFA-Net [8] (0.11 M parameters, 4.46 GFLOPs) and LightVesselNet [9] (75 K parameters; Dice 0.8070 on DRIVE and 0.8072 on STARE). They report parameters, memory size and FLOPs, which are useful proxies, but they require trained weights and a deep-learning runtime, and they do not give a stage-by-stage measured execution analysis on screening-class CPUs or GPUs.

4) GPU and parallel retinal vessel segmentation: Argüello et al. [3] tailor a hybrid of global image filtering and contour tracing to CUDA and segment a DRIVE image in 14 ms and a STARE image in 18 ms on an NVIDIA GTX 680, with accuracy comparable to the state of the art. Ben Sayadia et al. [4] design a GPU strategy for eigenvalue computation and report speedups of 27 over a multithreaded CPU version, 16 over OpenCV and 8 over cuBLAS, with the eigenvalue step taking 0.017 s and thick-vessel segmentation about 0.039 s per image. These are the closest prior works, but [3] uses a different algorithm and [4] addresses one stage and thick vessels.

5) Parallel image-processing methodology and GPU vesselness outside retina: Song et al. [14] implement unsharp-masking enhancement in serial, OpenMP, CUDA and OpenCL, apply memory optimisations (local and constant memory, multi-point access) and test image sizes from 525×525 to $16{,}364 \times 8{,}182$. They report up to $16.71\times$ speedup over serial for OpenCL, OpenMP speedup tracking the core count (up to $4.54\times$), speedup flattening for larger images as host–device transfer grows, and identical output quality across implementations. This is the methodological template for our comparison, but it covers a single enhancement stage and no vessel analysis. Nahar et al. [15] accelerate Hessian, eigenvalue and vesselness computation on a GPU for 3-D mitochondrial networks, achieving $11\times$ and $30\times$ speedups with about 99.9% agreement, and observe that small inputs under-use the GPU because of fixed costs such as kernel-launch overhead. This supports the expectation that vesselness is GPU-friendly and that image size matters, but the setting is 3-D microscopy and the implementation is library-based.

C. Comparison of the Surveyed Work

Table I summarises the approach, reported results, advantages and limitations of each surveyed work.

D. Performance Comparison

The surveyed works use different hardware, images and metrics, so the figures in Table II are context and not a like-for-like ranking. Our own numbers will be measured on one machine under one protocol.

E. Research Gaps

RG1: No full-pipeline, three-platform study. Parallel retinal work has used other algorithms [3] or isolated one stage [4]. Based on this survey, we did not find a study that implements CLAHE, multi-scale Frangi and morphology in serial, OpenMP and CUDA and profiles each stage.

RG2: Generic parallel comparisons are not vessel-specific. The cross-platform comparison in [14] covers a single enhancement filter. Vesselness has a different mix of memory-bound, compute-bound and atomic-contention stages, so conclusions may not transfer.

RG3: Memory-hierarchy effects across Frangi scales are not analysed. The Gaussian kernel radius grows with σ, which changes data reuse and the benefit of shared-memory tiling. GPU vesselness work [4], [15] does not report how this varies with scale.

RG4: Resolution scalability and the CPU–GPU crossover for fundus images. Benchmark papers use DRIVE at about 0.33 megapixels. Works [14], [15] show that transfer and launch overheads matter at small sizes, but the crossover for a retinal vesselness pipeline, including transfer time, is not reported.

RG5: Efficiency of lightweight deep models is expressed through proxies. Papers [7]–[9] report parameters and FLOPs.

TABLE I
COMPARISON OF THE SURVEYED WORK

Ref. / year	Approach and algorithm	Reported results	Advantages	Limitations
[10] Chen et al., IEEE Access 2021	Review of deep-learning vessel segmentation; network architectures, obstacles, key aspects	Trend review; no new method	Consolidates architectures and open problems to 2021	Architecture-centric; execution cost on screening hardware is not the focus
[11] Sule, IEEE Access 2022	Survey of 110 papers (2016–2021); taxonomy of optimisation, regularisation, pooling, activation, transfer and ensemble techniques	Taxonomy and trends	Clear taxonomy of accuracy-improving techniques	Targets accuracy and generalisation; latency is not a taxonomy axis
[12] Mahapatra et al., Comput. Biol. Med. 2022	Improved multi-scale Frangi filter with MELPSO-optimised parameters; adaptive weighted spatial FCM clustering	Accuracy-oriented; improved vessel enhancement through parameter optimisation	Unsupervised; keeps interpretable Frangi core	Optimisation and clustering add cost; no parallel or real-time analysis
[13] Galdran et al., Sci. Rep. 2022	Minimalistic lightweight CNNs; cross-dataset performance study	State-of-the-art accuracy with small models	Small models, broad cross-dataset evidence	Supervised; needs annotated data, training and a DL runtime
[14] Song et al., Sci. Rep. 2022	Unsharp masking in serial, OpenMP, CUDA, OpenCL; local/constant memory optimisation	OpenCL up to $16.71\times$ vs serial; OpenMP up to $4.54\times$; speedup flattens for large images (transfer cost)	Same comparative method; image-size scaling; output equivalence checked	Single enhancement kernel; not vessel-specific or a full pipeline
[6] Gojić et al., arXiv 2023	Overview of recent deep vessel-segmentation methods and their metrics	Metric comparison across methods	Quick reference for deep-learning accuracy	Preprint; accuracy only
[7] Mehmood et al. (LVS-Net), arXiv 2024	Lightweight encoder-decoder with multi-scale blocks and focal modulation attention; DRIVE, CHASE_DB, STARE	0.71 M parameters, 29.60 GFLOPs, 2.74 MB	Small footprint, multi-scale design	Trained model; reports FLOPs rather than measured stage timings
[8] Mehmood et al. (LFA-Net), arXiv 2025	LiteFusion attention, Mamba-inspired; DRIVE, STARE, CHASE_DB	0.11 M parameters, 0.42 MB, 4.46 GFLOPs	Very small model for constrained settings	Trained model; no measured CPU/GPU stage analysis
[9] Sobhan and Jalil (LightVesselNet), arXiv 2026	75 K-parameter encoder-decoder with channel/spatial attention; five datasets	DRIVE: Dice 0.8070, sensitivity 0.8189. STARE: Dice 0.8072, sensitivity 0.8499	Edge-oriented; competitive with larger models	Learned model; efficiency expressed as accuracy vs parameters/GFLOPs
[15] Nahar et al., BMC Bioinf. 2026	GPU (CuPy) Hessian, eigenvalue and vesselness with FFT convolution for 3-D mitochondria	$11\times$ and $30\times$ speedup; about 99.9% agreement	Shows vesselness suits GPUs; notes small inputs under-use the GPU	3-D microscopy; library-based Python; no OpenMP comparison
[4] Ben Sayadia et al., J. Electron. Imaging 2020	GPU strategy for eigenvalue computing; applied to retinal thick-vessel segmentation	$27\times$ vs multithreaded CPU, $16\times$ vs OpenCV, $8\times$ vs cuBLAS; about 0.039 s per image	Directly relevant Hessian-eigenvalue stage on GPU	One stage; thick vessels only; not a full CLAHE–Frangi–morphology pipeline
[3] Argüello et al., J. Real-Time Image Process. 2018	Global filtering plus contour tracing, tailored to CUDA	DRIVE 14 ms, STARE 18 ms (GTX 680); accuracy comparable to state of the art	Real-time; strong accuracy/speed ratio	Different algorithm (not Frangi); single GPU; no OpenMP comparison reported

A classical pipeline with measured throughput on commodity hardware is a useful, training-free baseline for deployment discussions.

F. Motivation for the Proposed Work

Taken together, the surveyed work shows that (i) vesselness filtering maps well to GPUs, (ii) OpenMP and CUDA behave differently as image size grows, and (iii) lightweight deep models address deployment through model size. What is missing is a careful, reproducible comparison of serial, OpenMP and CUDA versions of one complete classical retinal pipeline that explains the results through the memory hierarchy and stage characteristics. This project fills that gap and delivers a verified, documented code base and an analysis that a screening programme could use when choosing hardware.

III. PROJECT OBJECTIVES

The project has seven measurable objectives, listed with their success measures in Table III. In summary, the pipeline is implemented in serial C, parallelised with OpenMP and with CUDA (naive global-memory and shared-memory tiled kernels), verified against the serial output, profiled stage by stage, and studied for scalability with image resolution from

TABLE II
REPORTED PERFORMANCE OF CLOSELY RELATED WORK

Work	Task	Reported speed or quality
[3] Argüello et al.	Vessel tree, DRIVE / STARE, GTX 680	14 ms / 18 ms per image
[4] Ben Sayadia et al.	Eigenvalue stage; thick-vessel segmentation	0.017 s (eigenvalues); about 0.039 s (segmentation)
[14] Song et al.	Unsharp masking, 525×525 to $16{,}364 \times 8{,}182$	OpenCL up to $16.71\times$; OpenMP up to $4.54\times$; GPU speedup flattens above about 750×750
[15] Nahar et al.	3-D vesselness segmentation on GPU	$11\times$ (yeast), $30\times$ (lung) over CPU
[9] Sobhan and Jalil	DRIVE / STARE accuracy reference	Dice 0.8070 / 0.8072 with 75 K parameters
This project (planned)	Full pipeline, DRIVE and upscaled variants	To be measured: per-stage time, speedup, efficiency, crossover resolution, images/s, Dice

TABLE III
PROJECT OBJECTIVES AND SUCCESS MEASURES

No.	Objective	Success measure
O1	Implement the full pipeline in serial C and validate segmentation quality on DRIVE (parameters tuned on the 20 training images; results on the 20 test images; STARE as a cross-dataset check)	Dice, sensitivity, specificity and accuracy reported; cross-checked against a reference implementation (scikit-image Frangi) on sample images
O2	Parallelise all stages with OpenMP and CUDA	Both versions run end to end on all test images
O3	Verify equivalence with the serial output	Mask agreement of at least 99.9% of pixels and maximum vesselness difference within a stated tolerance; exact equality in a no-FMA test build
O4	Per-stage profiling on each platform	Time, speedup, efficiency and Karp–Flatt serial fraction per stage; bottleneck identified
O5	Shared-memory tiling analysis for the Gaussian-derivative stage	Naive vs tiled time and effective bandwidth for $\sigma = 1$ to 4 and for several tile sizes
O6	Resolution scalability study ($1\times$, $2\times$, $4\times$, $8\times$)	Speedup-versus-size curves; crossover resolution with and without transfer time
O7	Throughput and deployment view	Images per second on the test hardware; comparison with timings reported in [4] and [3]

$1\times$ to $8\times$, including the OpenMP-to-CUDA crossover and host–device transfer cost.

IV. PROPOSED METHODOLOGY

All three implementations execute the same algorithm so that differences in time can be attributed to the platform, not to the method. The pipeline is: green-channel extraction, CLAHE on an 8×8 tile grid, multi-scale Frangi vesselness, thresholding followed by binary opening and closing, evaluation against expert ground truth, and benchmarking with per-stage timers, thread and tile-size sweeps, resolution scaling and repeated runs. The stages are detailed below.

A. Datasets and Evaluation Protocol

DRIVE [5] contains 40 colour fundus images (565×584 pixels), split into 20 training and 20 test images, with manual vessel segmentations and field-of-view (FOV) masks. STARE [16] contains 20 images (700×605) and is used as a cross-dataset check with parameters fixed from DRIVE. All metrics are computed inside the FOV. The CLAHE clip limit, the Frangi parameters β and c, and the global threshold are tuned on the DRIVE training images only.

B. Pipeline Algorithms

Stage 1, green channel. $G(x, y)$ is taken from the RGB image, since vessels have the highest contrast in this channel.

Stage 2, CLAHE [17]. The image is divided into an 8×8 grid of tiles. For each tile a 256-bin histogram is built, clipped at the clip limit with the excess redistributed, and converted to a lookup table by the cumulative distribution. Each pixel output is a bilinear blend of the lookup tables of the four nearest tiles.

Stage 3, multi-scale Frangi filter [2]. For each scale σ, the scale-normalised Hessian is built from Gaussian-derivative responses of the CLAHE output I:

$$H(\sigma) = \sigma^2 \begin{bmatrix} I_{xx} & I_{xy} \\ I_{xy} & I_{yy} \end{bmatrix} \tag{1}$$

Here I_{xx}, I_{xy} and I_{yy} are the Gaussian second-derivative responses of I at scale σ. The two eigenvalues have a closed form, so no library is needed:

$$\lambda_{1,2} = \tfrac{1}{2}\left(I_{xx} + I_{yy}\right) \pm \tfrac{1}{2}\sqrt{\left(I_{xx} - I_{yy}\right)^2 + 4I_{xy}^2} \tag{2}$$

with $|\lambda_1| \le |\lambda_2|$. Vessels are dark on a bright background in the green channel, so the vesselness is non-zero only when $\lambda_2 > 0$:

$$R_b = \frac{|\lambda_1|}{|\lambda_2|}, \qquad S = \sqrt{\lambda_1^2 + \lambda_2^2} \tag{3}$$

$$V_\sigma = \exp\!\left(-\frac{R_b^2}{2\beta^2}\right)\left[1 - \exp\!\left(-\frac{S^2}{2c^2}\right)\right] \tag{4}$$

$$V(x, y) = \max_{\sigma \in \{1,2,3,4\}} V_\sigma(x, y) \tag{5}$$

Each second-derivative image is computed with separable one-dimensional Gaussian-derivative kernels of radius $r = \lceil 3\sigma \rceil$, giving six 1-D passes per scale (three row passes, three column passes).

Stage 4, threshold and morphology. V is thresholded with a global value tuned on the training set, then binary opening (to remove isolated speckle) and closing (to bridge small gaps) with a 3×3 structuring element are applied.

TABLE IV
PARALLEL PROGRAMMING MODELS

Model	Role in the project	What will be explored
Serial C	Correctness reference and timing baseline	`gcc -O3`, single thread; golden output for all comparisons
OpenMP	Shared-memory parallelism on multicore CPUs	`parallel for` / `collapse(2)` over pixels, tiles and scales; thread sweep 1 to the number of logical cores; scheduling and thread-affinity effects
CUDA	Data-parallel execution on an NVIDIA GPU	One thread per pixel; naive global-memory kernels versus shared-memory tiled kernels with halo; constant memory for filter weights; block-size and tile-size sweep; transfer cost
Hybrid (optional)	Stretch goal only	OpenMP threads feeding several CUDA streams for batch processing, if time permits

Stage 5, evaluation. True and false positives and negatives are counted inside the FOV and converted to the metrics in Section IV-F.

C. Parallel Programming Models

Table IV lists the programming models used, with OpenMP [18] for shared-memory CPU parallelism and CUDA [19] for data-parallel execution on an NVIDIA GPU.

D. Parallel Design by Stage

Table V gives the OpenMP and CUDA design for each stage together with the factor expected to limit it. For a separable pass with block width B_x and radius r, a tiled kernel loads $(B_x + 2r)$ elements per row for B_x outputs, whereas a naive kernel reads $2r + 1$ elements per output. For $\sigma = 4$ ($r = 12$) and $B_x = 16$ this is about 2.5 against 25 global reads per output pixel, a 10-fold reduction in read traffic. This is a design estimate to be confirmed by measurement, since caches also reduce the naive kernel's real traffic.

E. Experimental Setup and Measurement Protocol

- **Hardware:** CPU model, physical and logical cores, RAM, GPU model, VRAM, driver and CUDA versions will be recorded from the test machine and listed in the report.
- **Software:** C with `gcc -O3 -fopenmp`; CUDA with `nvcc -O3` and the correct compute-capability flag; `stb_image` for image I/O; Python for plotting; Git for version control.
- **Timing:** 5 warm-up runs then 30 timed runs per configuration; mean, standard deviation and median reported. CPU timing with `omp_get_wtime`, GPU timing with CUDA events. File I/O is excluded. Kernel-only and end-to-end times (including host-to-device and device-to-host transfers) are reported separately.
- **OpenMP:** thread counts 1, 2, 4, 8 and up to the number of logical cores; fixed thread affinity (`OMP_PROC_BIND`, `OMP_PLACES`); `schedule(static)` versus `dynamic` compared for CLAHE.
- **CUDA:** block sizes 8×8, 16×16 and 32×8; tile sizes varied for the tiled kernels; effective bandwidth computed from bytes moved and kernel time.
- **Scalability:** each DRIVE image is bilinearly resampled to $1\times$, $2\times$, $4\times$ and $8\times$ linear size (about 0.33 to 21 megapixels) and the same pipeline is timed.

F. Evaluation Metrics

The metrics used are defined in Table VI.

G. Correctness Verification

The serial output is the golden reference. Parallel results are compared with it stage by stage. Bit-exact equality cannot be assumed in general: fused multiply-add on GPUs and different summation order in reductions change floating-point rounding. We therefore report (i) the percentage of identical mask pixels and the maximum absolute vesselness difference, and (ii) an exact-equality test using a build with fused multiply-add disabled (`nvcc -fmad=false`, `gcc -ffp-contract=off`) for the deterministic stages. The serial implementation is additionally cross-checked on sample images against the scikit-image Frangi filter.

H. Risks and Mitigation

Table VII lists the main risks and how they will be handled.

V. EXPECTED OUTCOMES AND HYPOTHESES

The deliverables are:

- Documented C/CUDA source code for the three implementations, with a build script and a benchmark harness that writes CSV logs.
- Per-stage timing, speedup and efficiency tables; scalability plots; shared-memory versus global-memory comparison; correctness and accuracy tables.
- A final report in Springer Nature and IEEE conference formats and a paper draft for submission.

The following hypotheses will be tested and are not claimed in advance.

H1. OpenMP speedup grows sub-linearly with thread count and is limited by physical cores and memory bandwidth. A related OpenMP study reports speedup close to the core count [14].

H2. CUDA speedup grows with image size until host–device transfer dominates. At the native DRIVE size (565×584) the GPU may be under-used, so a crossover resolution is expected [14], [15].

H3. Shared-memory tiling helps the Gaussian-derivative stage more as σ grows, because the number of redundant global reads per output pixel rises with kernel radius. It should help the arithmetic-heavy eigenvalue stage very little.

H4. The classical pipeline will not match the best lightweight deep models in Dice (for example, 0.8070 on DRIVE for LightVesselNet [9]). The contribution is the parallel performance analysis, and accuracy will be reported honestly alongside it.

TABLE V
PARALLEL DESIGN BY PIPELINE STAGE

Stage	OpenMP	CUDA	Hypothesised limiting factor
Green channel	`parallel for` over pixels	1 thread per pixel; coalesced access	Memory bandwidth
CLAHE	`parallel for` over tiles; per-thread histograms	1 block per tile; histogram in shared memory with atomics (sub-histograms if contention); 1 thread per pixel for interpolation	Atomic contention and scattered bin updates
Gaussian-derivative passes	`collapse(2)` over rows and columns; scale loop parallel or sequential	Naive: reads from global memory. Tiled: block loads tile plus halo of radius r into shared memory; weights in constant memory	Memory bandwidth; global reads per output fall from about $(2r+1)$ to about $(1+2r/B_x)$ for tile width B_x
Hessian, eigenvalues, vesselness	`parallel for` over pixels, fused with scale maximum	1 thread per pixel per scale; streams per scale as an extra experiment	Arithmetic (square root, division, exponentials)
Threshold, opening, closing	`parallel for` over rows	3×3 min/max with shared-memory tile and 1-pixel halo	Memory bandwidth; low reuse
Metrics	`reduction` clause for TP/FP/TN/FN	Block-level reduction then atomic add	Negligible; checks correctness

TABLE VI
EVALUATION METRICS

Metric	Definition
Dice coefficient	$2\,\mathrm{TP}/(2\,\mathrm{TP}+\mathrm{FP}+\mathrm{FN})$
Sensitivity, specificity, accuracy	$\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$; $\mathrm{TN}/(\mathrm{TN}+\mathrm{FP})$; $(\mathrm{TP}+\mathrm{TN})/(\mathrm{TP}+\mathrm{TN}+\mathrm{FP}+\mathrm{FN})$
Speedup and efficiency	$S(p) = T(1)/T(p)$; $E(p) = S(p)/p$
Karp–Flatt serial fraction [20]	$e = (1/S - 1/p)/(1 - 1/p)$; a rising e with p indicates overhead, not a fixed serial part
Throughput	Images per second $= 1/$ end-to-end time per image
Crossover resolution	Smallest image size at which CUDA end-to-end time is below OpenMP time

TABLE VII
RISKS AND MITIGATION

Risk	Mitigation
Small images under-use the GPU	Resolution sweep and a batch-of-images experiment; report both kernel-only and end-to-end times
Atomic contention in CLAHE histograms	Shared-memory histograms and per-warp sub-histograms; compare variants
Floating-point differences between platforms	Tolerance-based comparison plus the no-FMA exact test
GPU availability	Use a lab or personal NVIDIA GPU; if only a cloud GPU is available, report it as a separate platform and do not mix it with local CPU numbers
Limited time	Core scope is DRIVE with serial, OpenMP, CUDA naive and tiled; STARE, multi-stream and hybrid experiments are stretch goals

VI. IMPLEMENTATION PLAN AND WORK DISTRIBUTION

The schedule in Table VIII is compressed to the time remaining before the final submission and should be adjusted to the dates approved by the faculty mentor. Table IX gives the proposed work distribution for a team of four; roles can be merged for a smaller team.

TABLE VIII
TENTATIVE IMPLEMENTATION PLAN

Phase and window	Activities	Deliverable / review
1. Setup and serial baseline (3–7 Oct)	Download DRIVE/STARE; image I/O; green channel, CLAHE, Frangi, threshold, morphology, metrics in serial C; tune parameters on training images	Working serial pipeline with Dice on DRIVE test set
2. OpenMP and naive CUDA (8–14 Oct)	OpenMP versions of all stages and thread sweep; CUDA global-memory kernels for all stages; timing harness	Progress Review 3: all three versions running, first timing tables
3. Optimisation and experiments (15–21 Oct)	Shared-memory tiled kernels; tile and block-size sweep; resolution scaling $1\times$–$8\times$; transfer-time study; profiling	Complete result set, graphs, correctness checks
4. Report, demo, paper (22–28 Oct)	Analysis and discussion; Springer and IEEE reports; presentation; demo script; paper draft	Final report, presentation and demonstration

VII. CONCLUSION

This proposal defines a reproducible comparison of serial, OpenMP and CUDA implementations of one complete classical retinal vessel segmentation pipeline on the DRIVE benchmark. The survey shows that parallel retinal work has so far used other algorithms or isolated single stages, which leaves stage-by-stage profiling, memory-hierarchy effects across Frangi scales and the CPU–GPU crossover resolution unreported. The planned implementations, correctness checks and measurement protocol are intended to fill these gaps and to give screening programmes evidence for choosing hardware.

TABLE IX
PROPOSED WORK DISTRIBUTION

Member	Responsibility
Member 1	Serial pipeline, parameter tuning, accuracy evaluation, ground-truth comparison
Member 2	OpenMP versions, thread-scaling experiments, timing harness and CSV logging
Member 3	CUDA naive kernels, CLAHE and morphology kernels, transfer-time study
Member 4	CUDA shared-memory Gaussian-derivative kernels, tile-size and resolution experiments
All	Literature survey, plots, report writing, presentation and demonstration

REFERENCES

[1] International Diabetes Federation, *IDF Diabetes Atlas*, 11th ed. Brussels, Belgium: IDF, 2025. [Online]. Available: https://diabetesatlas.org

[2] A. F. Frangi, W. J. Niessen, K. L. Vincken, and M. A. Viergever, "Multiscale vessel enhancement filtering," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI'98)*, Lecture Notes in Computer Science, vol. 1496. Berlin, Germany: Springer, 1998, pp. 130–137.

[3] F. Argüello, D. L. Vilariño, D. B. Heras, and A. Nieto, "GPU-based segmentation of retinal blood vessels," *J. Real-Time Image Process.*, vol. 14, no. 4, pp. 773–782, 2018, doi: 10.1007/s11554-014-0469-z.

[4] S. Ben Sayadia, Y. Elloumi, M. Akil, and M. H. Bedoui, "Efficient graphical-processor-unit parallelization algorithm for computing Eigen values," *J. Electron. Imaging*, vol. 29, no. 6, Art. no. 063008, Dec. 2020, doi: 10.1117/1.JEI.29.6.063008.

[5] J. Staal, M. D. Abràmoff, M. Niemeijer, M. A. Viergever, and B. van Ginneken, "Ridge-based vessel segmentation in color images of the retina," *IEEE Trans. Med. Imaging*, vol. 23, no. 4, pp. 501–509, Apr. 2004.

[6] G. Gojić, O. Kundačina, D. Mišković, and D. Dragan, "Overview of deep learning methods for retinal vessel segmentation," arXiv:2306.06116, Jun. 2023.

[7] M. Mehmood, S. Iqbal, T. M. Khan, I. Spence, and M. Fahim, "LVS-Net: A lightweight vessels segmentation network for retinal image analysis," arXiv:2412.05968, Dec. 2024.

[8] M. Mehmood, I. Spence, and M. Fahim, "LFA-Net: A lightweight network with LiteFusion attention for retinal vessel segmentation," arXiv:2509.21738, Sep. 2025.

[9] S. Sobhan and F. Jalil, "LightVesselNet: An ultra-lightweight sub-100K parameter network for retinal blood vessel segmentation," arXiv:2606.05354, Jun. 2026.

[10] C. Chen, J. H. Chuah, R. Ali, and Y. Wang, "Retinal vessel segmentation using deep learning: A review," *IEEE Access*, vol. 9, pp. 111985–112004, 2021, doi: 10.1109/ACCESS.2021.3102176.

[11] O. O. Sule, "A survey of deep learning for retinal blood vessel segmentation methods: Taxonomy, trends, challenges and future directions," *IEEE Access*, vol. 10, pp. 38202–38236, 2022, doi: 10.1109/ACCESS.2022.3163247.

[12] S. Mahapatra, S. Agrawal, P. K. Mishro, and R. B. Pachori, "A novel framework for retinal vessel segmentation using optimal improved Frangi filter and adaptive weighted spatial FCM," *Comput. Biol. Med.*, vol. 147, Art. no. 105770, Aug. 2022, doi: 10.1016/j.compbiomed.2022.105770.

[13] A. Galdran *et al.*, "State-of-the-art retinal vessel segmentation with minimalistic models," *Sci. Rep.*, vol. 12, 2022, doi: 10.1038/s41598-022-09675-y.

[14] Y. Song, C. Li, S. Xiao, H. Xiao, and B. Guo, "Unsharp masking image enhancement the parallel algorithm based on cross-platform," *Sci. Rep.*, vol. 12, Art. no. 20175, Nov. 2022, doi: 10.1038/s41598-022-21745-9.

[15] S. Nahar, Z. Wang, E. Arkfeld, G. McMahon, H. Hakozaki, and J. Schöneberg, "GPU-accelerated MitoGraph for high-throughput three-dimensional mitochondrial morphology analysis," *BMC Bioinformatics*, vol. 27, Art. no. 206, 2026, doi: 10.1186/s12859-026-06441-z.

[16] A. Hoover, V. Kouznetsova, and M. Goldbaum, "Locating blood vessels in retinal images by piecewise threshold probing of a matched filter response," *IEEE Trans. Med. Imaging*, vol. 19, no. 3, pp. 203–210, Mar. 2000.

[17] K. Zuiderveld, "Contrast limited adaptive histogram equalization," in *Graphics Gems IV*, P. S. Heckbert, Ed. San Diego, CA, USA: Academic Press, 1994, pp. 474–485.

[18] L. Dagum and R. Menon, "OpenMP: An industry standard API for shared-memory programming," *IEEE Comput. Sci. Eng.*, vol. 5, no. 1, pp. 46–55, Jan.–Mar. 1998.

[19] J. Nickolls, I. Buck, M. Garland, and K. Skadron, "Scalable parallel programming with CUDA," *ACM Queue*, vol. 6, no. 2, pp. 40–53, 2008.

[20] A. H. Karp and H. P. Flatt, "Measuring parallel processor performance," *Commun. ACM*, vol. 33, no. 5, pp. 539–543, May 1990.